

Parallel Computing

Project 1: Poisson Problem in OpenMP

Vintila Andrei Dragos $\Leftarrow$ s1098027

May 12, 2025

This report evaluates a red-black successive over-relaxation (SOR) solver for the Poisson problem with periodic boundary conditions, in a sequential version (`sor_seq.c`) and an OpenMP version (`sor_omp.c`). Both are based on the starter code provided by the course (`sor.c`); our contributions are the tiling of the loops, the OpenMP parallelization and the measurements.

Computer Specifications

Processor	11th Gen Intel® Core™ i7-1165G7
Base Frequency	2.80 GHz
Max Turbo Frequency	4.70 GHz
Current Frequency Range	0.40 GHz (idle) - 4.70 GHz (max)
CPU Scaling	20%
Memory Capacity	16 GB (2 × 8 GB DDR4)
Memory Speed	3200 MHz
Core Configuration	4 physical cores, 8 threads
L2 Cache	5 MB (1.25 MB per core)
L3 Cache	12 MB (shared)

From these specifications, we can compute the theoretical peak floating-point performance (GFLOPS) for a single core operating at maximum turbo frequency:

$$16 \text{ FLOPs/cycle} \cdot 4.7 \cdot 10^9 \text{ cycles/sec} = 75.2 \text{ GFLOPS}$$

The theoretical bandwidth of dual-channel DDR4-3200 memory is 51.2 GB/s.

Methodology

- **Compiler and flags.** GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), with `-O3 -march=native` for both versions, `-fopenmp` for the OpenMP version, linked with `-lm`:

```
gcc -O3 -march=native -o sor_seq sor_seq.c -lm
gcc -O3 -march=native -fopenmp -o sor_omp sor_omp.c -lm
```

- **Runs.** For $N \in \{64, 128, 256, 512, 1024\}$ and $T \in \{1, 2, 4, 8\}$:

```
./sor_seq N 1000
OMP_NUM_THREADS=T ./sor_omp N 1000
```

- **Repetitions.** Each program solves the problem 10 times, resetting the grid to zero before every solve. We report the mean and the sample standard deviation of the time per solve, written as mean $\pm$ standard deviation.

- **Iterations.** The solver runs for the number of iterations given by the convergence formula in the starter code, capped at the `MAX_ITER` argument (1000 here). The 64×64 grid needs only 740 iterations; all other grids hit the cap of 1000.
- **Machine.** All measurements were taken on the laptop described above, not on the compute node of the course, with a desktop session and other programs open. The machine has 4 physical cores, so the 8-thread runs use hyper-threads.
- **Metrics.** As in the starter code, bandwidth is $3 \cdot 8N^2 \cdot \text{iterations}/t$ bytes per second and the compute rate is $11 \cdot N^2 \cdot \text{iterations}/t$ FLOPs per second, where t is the mean time per solve.

Task 1. Sequential Performance Evaluation

Table 1 shows the sequential solver for all grid sizes. “Both arrays” is the memory taken by the solution and the right-hand side together, including ghost cells.

Table 1: Sequential performance (1000 iterations, 740 for $N = 64$)

Grid	Both arrays (MB)	Time (ms)	Bandwidth (GB/s)	Compute rate (GFLOPS)	Per cell update (ns)
64×64	0.07	3.70 ± 2.42	19.7	9.0	1.22
128×128	0.27	15.4 ± 0.6	25.6	11.7	0.94
256×256	1.07	57.0 ± 0.8	27.6	12.6	0.87
512×512	4.23	289 ± 13	21.7	10.0	1.10
1024×1024	16.84	2503 ± 29	10.1	4.6	2.39

Relating these numbers to the hardware leads to the following conclusions:

- **Small grids are faster per cell because they stay in cache.** Up to 256×256 both arrays fit in the 1.25 MB L2 cache of one core, and at 512×512 they fit in the 12 MB L3 cache. Across these sizes one cell update costs 0.87 to 1.22 ns. At 1024×1024 the two arrays take 16.8 MB, more than the L3 cache, and the cost per update rises to 2.39 ns, more than twice the cost at 512×512 .
- **The bandwidth metric only measures memory traffic for the largest grid.** For the grids that fit in cache, the 20 to 28 GB/s in the table is data moved between the core and its caches, so it cannot be compared with the DDR4 limit. For 1024×1024 , a single thread reaches 10.1 GB/s, about a fifth of the theoretical 51.2 GB/s.
- **The compute rate is far below the theoretical peak.** The best value, 12.6 GFLOPS at 256×256 , is 17% of the 75.2 GFLOPS peak. The peak assumes that every cycle performs full-width vector arithmetic, which a stencil that updates every second cell from five loaded values does not reach.
- **The 64×64 measurement is noisy.** Its standard deviation is 65% of the mean, so this row is unreliable. The OpenMP version with one thread measured 3.10 ± 0.08 ms on the same grid (23.4 GB/s), which is in line with the other grids that fit in cache.

Convergence. The relative error of a converged solution is the discretization error of the five-point stencil, approximately $h^2/12$ with $h = 2\pi/N$, so it becomes four times smaller every time N doubles. Larger grids need many more iterations to get there: the formula in the starter code asks for 740 iterations at $N = 64$, 1946 at $N = 128$, 4803 at $N = 256$ and 11411 at $N = 512$. This matches what we measured:

- The 64×64 grid improves from $3.9 \cdot 10^{-2}$ after 100 iterations to $8.0 \cdot 10^{-4}$, and stays there for any larger `MAX_ITER`, because the solver stops after 740 iterations and $8.0 \cdot 10^{-4}$ is the discretization error of this grid.

- The 512×512 grid improves from $6.9 \cdot 10^{-1}$ after 100 iterations to $1.9 \cdot 10^{-2}$ after 1000 and $1.3 \cdot 10^{-5}$ after 10000, close to its discretization error.
- With the cap of 1000 iterations used for the timings, the grids from 256×256 upwards are therefore not fully converged (relative errors $2.9 \cdot 10^{-4}$, $1.9 \cdot 10^{-2}$ and $1.4 \cdot 10^{-1}$). This does not affect the timings, since every iteration does the same amount of work.
- For a 10240×10240 grid the relative error is still 1.000 after one or two iterations. That is expected: the formula asks for about 380000 iterations at this size.

Tiling. We divided the grid operations into 64×64 tiles ($64 \cdot 64 \cdot 8 \text{ B} = 32 \text{ KB}$ per tile) to improve cache use. The expected gain is small, because within one sweep every cell is still visited exactly once, so tiling does not add reuse of cached data. In an earlier measurement with 100 iterations at 512×512 , the tiled version was about 9% faster than the starter code. We did not repeat this comparison for the measurements in this report, so it is indicative only.

Task 2: OpenMP Performance Evaluation

The OpenMP version is parallelized as follows:

- **SOR sweeps.** Each iteration consists of a sweep over the black cells followed by a sweep over the red cells. A cell of one color only reads cells of the other color, so all cells of one color can be updated at the same time without data races. Each sweep is a `parallel for` over rows of 64×64 tiles.
- **Scheduling.** The sweeps use `schedule(dynamic)`; the initialization loops use `schedule(static)`. Every row of tiles holds the same amount of work, so static scheduling would be an equally valid choice for the sweeps. We did not compare the two.
- **Other loops.** Initialization uses `collapse(2)` on the perfectly nested tile loops, and the L_∞ norm uses `reduction(max:maximum)`.
- **Ghost cells.** The ghost cell update runs serially. It copies only $2(m+n)+4$ cells, so opening a parallel region for it costs more than the copy.
- **Thread count.** The number of threads is taken from `OMP_NUM_THREADS`, with `omp_set_dynamic(0)` so that it stays fixed during a run.
- **Limit on parallelism.** Because the parallel loop runs over rows of tiles, a grid of size N has only $N/64$ units of work to hand out per sweep: 1 at $N = 64$, 2 at $N = 128$, 4 at $N = 256$, 8 at $N = 512$ and 16 at $N = 1024$. The 64×64 grid can therefore not be sped up at all by this code, and the 128×128 grid by at most a factor of two.

For a 512×512 grid with 1000 iterations, 4 threads compare with the sequential version as follows:

Metric	Sequential	OpenMP, 4 threads	Factor
Time per solve	$289 \pm 13 \text{ ms}$	$163 \pm 9 \text{ ms}$	$1.78\times$ faster
Bandwidth	21.7 GB/s	38.7 GB/s	$1.78\times$
Compute rate	10.0 GFLOPS	17.7 GFLOPS	$1.78\times$
Relative error	$1.861 \cdot 10^{-2}$	$1.861 \cdot 10^{-2}$	(unchanged)

Task 3: Statistics

Table 2: Time per solve in ms, mean $\pm$ standard deviation over 10 solves

Grid	Sequential	1 thread	2 threads	4 threads	8 threads
64	3.70 ± 2.42	3.10 ± 0.08	5.32 ± 0.07	5.38 ± 0.12	23.1 ± 15.1
128	15.4 ± 0.6	14.4 ± 1.0	15.0 ± 0.3	18.6 ± 0.2	37.8 ± 18.0
256	57.0 ± 0.8	68.9 ± 2.8	53.0 ± 4.6	47.8 ± 5.0	105 ± 59
512	289 ± 13	296 ± 24	194 ± 8	163 ± 9	209 ± 69
1024	2503 ± 29	2537 ± 139	1556 ± 95	877 ± 29	1073 ± 125

Table 3: Speedup over the sequential version

Grid	1 thread	2 threads	4 threads	8 threads
64	1.19	0.70	0.69	0.16
128	1.07	1.02	0.83	0.41
256	0.83	1.08	1.19	0.54
512	0.98	1.49	1.78	1.38
1024	0.99	1.61	2.85	2.33

Table 4: Strong scaling efficiency (%)

Grid	1 thread	2 threads	4 threads	8 threads
64	119	35	17	2
128	107	51	21	5
256	83	54	30	7
512	98	74	44	17
1024	99	80	71	29

Table 5: Weak scaling: four times the cells on four times the threads, 1000 iterations each

Grid, 1 thread	Time (ms)	Grid, 4 threads	Time (ms)	Efficiency (%)
128×128	14.4 ± 1.0	256×256	47.8 ± 5.0	30
256×256	68.9 ± 2.8	512×512	163 ± 9	42
512×512	296 ± 24	1024×1024	877 ± 29	34

Where:

- **Speedup** $S_p = \frac{T_{\text{seq}}}{T_p}$: the time of the sequential program divided by the time of the OpenMP program with p threads, on the same grid.
- **Strong scaling efficiency** $= \frac{S_p}{p} \times 100\%$: the fraction of perfect scaling achieved when the problem size is fixed and threads are added.
- **Weak scaling efficiency** $= \frac{T_1(N)}{T_4(2N)} \times 100\%$: the time of the OpenMP program with 1 thread on an $N \times N$ grid divided by its time with 4 threads on a $2N \times 2N$ grid. The work per thread is the same in both runs, so perfect scaling would give 100%.

Task 3: Analysis

- **Small grids: parallel overhead dominates.** For 64×64 and 128×128 , adding threads makes the program slower. One iteration on the 64×64 grid takes about $4 \mu\text{s}$ with one

thread, $7\ \mu\text{s}$ with two or four threads and $31\ \mu\text{s}$ with eight, although the work is the same. The cost of starting two parallel regions per iteration and distributing the work is larger than the work itself, and as noted in Task 2 this grid has only one unit of work per sweep, so the extra threads cannot contribute.

- **Large grids: useful speedup on the physical cores.** The speedup grows with the grid size. The best result is $2.85\times$ with 4 threads on the 1024×1024 grid, an efficiency of 71%; at 512×512 it is $1.78\times$ (44%).
- **Eight threads are slower than four on every grid.** The machine has 4 physical cores, so threads 5 to 8 share cores with the first four. These runs are also the least stable, with standard deviations between 12% and 65% of the mean, so the 8-thread column only shows that using the hyper-threads does not help.
- **Overhead of OpenMP with one thread.** With one thread the OpenMP version is within the measurement noise of the sequential version at 128, 512 and 1024. At 256×256 it takes 21% longer. The 119% at 64×64 comes from the noisy sequential measurement and is not a real gain.
- **Memory bandwidth.** The 1024×1024 grid is the only one that does not fit in the L3 cache. There the bandwidth metric goes from 10.1 GB/s sequentially to 28.7 GB/s with 4 threads, against a theoretical limit of 51.2 GB/s. An efficiency of 71% rather than 100% is consistent with the threads starting to compete for memory bandwidth, but our measurements do not prove it: within the range we measured, the largest grid scales best. Grids of 2048×2048 and larger would be needed to see whether the speedup flattens.
- **Weak scaling.** The weak scaling efficiency is between 30% and 42%. The step from 512×512 to 1024×1024 also crosses from the L3 cache into main memory, where every cell update is more than twice as expensive (Table 1), so that row combines the parallel overhead with the cache effect.